1 Other (More Specific) Kinds of Solvers and Encodings

Why talk about these?

- To give you an idea of what's out there, and different kinds of encodings.
- To give you something to compare to.

We will talk about:

- **Integer programs**
- SAT solvers
- Stochastic and local search (next time)

And how we might compare the performance of different solvers or techniques.

1.1 Integer programming

We have in a sense already done some integer programming, just encoded as CSPs in MiniZinc style.

An integer program is a mathematical optimisation in which the variables are integers (if you've seen linear programming before, it's a linear program where we restrict variables to integer values).

- Often people actually mean integer *linear* programming, where the constraints are linear.
- If some of the variables are not integers, we have *mixed-integer* programming (MIP).
- If the integer domains are restricted to subsets of $\{0, 1\}$ then we call it zero-one, or binary, integer programming.

How do we encode a problem as an ILP? Learn by examples.

1.1.1 Knapsack example

We are a child running away from home, and are packing a knapsack of snacks. Each snack has a cost in terms of backpack space (or weight maybe), and a desirability. We want to optimise our desirability subject to space. We have:

- apples (a): cost 3, gain 4;
- lettuces (l): cost 7, gain 2;

- carrots (c): cost 4, gain 2.

So we want to maximise

$$4a + 2l + 2c$$

subject to

$$3a + 7l + 4c \leq \text{capacity}.$$

1.1.2 Graph colouring example

We have a graph $G = (V, E)$ and we want to colour it with a minimum number of colours. We need a number of variables and constraints.

(Boolean) Variables:

- For each vertex i and colour j:
  - $c_{i,j}$ is 1 if vertex i is colour j, 0 otherwise;
  - w_j is 1 if we use colour j, 0 otherwise.
- The domain of i is the set of vertices, and the domain of j is the set of colours.

Objective function:

$$\min \sum_j w_j$$

Subject to constraints:

- Every node gets exactly one colour:

$$\forall i \in V, \quad \sum_j c_{i,j} = 1.$$

- Every edge has different-coloured ends:

$$\forall (u, v) \in E,\ j \in C, \quad c_{u,j} + c_{v,j} \leq 1.$$

- We record when we use a colour:

$$\forall i \in V,\ j \in C, \quad c_{i,j} \leq w_j.$$

1.1.3 A nice example using OR-Tools

A worked example is available as a Colab notebook, adapted from this blog post.

That's from a student who was taking a course at Bonn – the deep-dive notes on that section are in these slides, if you're interested.

And if you're *really* keen, here's a related paper.

1.1.4 A PuLP in Python example

See this Colab notebook.

1.1.5 An ILP for firefighting

Firefighting problem: a fire breaks out on a graph $G = (V, E)$ at vertex r. On each turn we get to defend d vertices. Then the fire spreads to all non-burning, non-defended vertices that are adjacent to a burning vertex. Burning vertices burn forever, defended vertices are defended forever.

Questions:

- **Decision version:** can we save some specified proportion of the graph (or number of vertices)?
- **Optimisation version:** what is the maximum number of vertices we can save?

A few resources:

- a classic survey: A. Finbow and G. MacGillivray, "The Firefighter Problem: A survey of results, directions and questions," *Australasian Journal of Combinatorics*, 43 (2009), 57–77.
- a paper about ILP and heuristic techniques on this problem: `https://dl.acm.org/doi/10.1016/j.cor.2015.02.004`.

We can encode this as an ILP.

Some notation:

- $G = (V, E)$ is the graph;
- T is some maximum time;
- $N(x)$ is the neighbourhood of vertex x;
- we'll use t to denote times;
- r_i are the places the fire starts, for $1 \leq i \leq f$;
- f is the number of vertices where the fire starts;
- d is our budget of defenders.

Some variables and how we interpret them:

- $b_{x,t}$ gets 1 if vertex x is burned at or before t, 0 otherwise;
- $d_{x,t}$ gets 1 if vertex x is defended at or before t, 0 otherwise.

$$\text{Min} \sum_{x \in V} b_{x,T}$$

Subject to:

$$\begin{cases} b_{x,t} \geq b_{x,t-1} & x \in V, 1 \leq t \leq T \\ d_{x,t} \geq d_{x,t-1} & x \in V, 1 \leq t \leq T \\ b_{x,t} + d_{x,t} \geq b_{y,t-1} & x \in V, y \in N(x), 1 \leq t \leq T \\ b_{x,t} + d_{x,t} \leq 1 & x \in V, 1 \leq t \leq T \\ \sum_{y \in N(x)} b_{y,t-1} \geq b_{x,t} & x \in V, 1 \leq t \leq T \\ \sum_{x \in V} (d_{x,t} - d_{x,t-1}) \leq d & 1 \leq t \leq T \\ d_{x,0} = 0 & x \in V \\ b_{x,0} = \begin{cases} 1 & \text{if } x = r_i \text{ for some } i, 1 \leq i \leq f \\ 0 & \text{otherwise} \end{cases} & x \in V \\ b_{x,t}, d_{x,t} \in \{0,1\} & 1 \leq t \leq T \end{cases}$$

Figure 3: A 0-1 integer program for the Firefighter Problem

Figure 1: A 0-1 integer program for the Firefighter Problem.

The program above uses the classic notation, but as some students find it easier to parse programs that include names rather than b and d, below is the same program but with renamed variables:

$$\text{Min} \sum_{x \in V} \text{burn}_{x,T}$$

Subject to:

$$\begin{cases} \text{burn}_{x,t} \geq \text{burn}_{x,t-1} & x \in V,\ 1 \leq t \leq T \\ \text{defend}_{x,t} \geq \text{defend}_{x,t-1} & x \in V,\ 1 \leq t \leq T \\ \text{burn}_{x,t} + \text{defend}_{x,t} \geq \text{burn}_{y,t-1} & x \in V,\ y \in N(x),\ 1 \leq t \leq T \\ \text{burn}_{x,t} + \text{defend}_{x,t} \leq 1 & x \in V,\ 1 \leq t \leq T \\ \sum_{y \in N(x)} \text{burn}_{y,t-1} \geq \text{burn}_{x,t} & x \in V,\ 1 \leq t \leq T \\ \sum_{x \in V} (\text{defend}_{x,t} - \text{defend}_{x,t-1}) \leq \textit{defense-budget} & 1 \leq t \leq T \\ \text{defend}_{x,0} = 0 & x \in V \\ \text{burn}_{x,0} = \begin{cases} 1 & \text{if } x = r_i \text{ for some } i,\ 1 \leq i \leq f \\ 0 & \text{otherwise} \end{cases} & x \in V \\ \text{burn}_{x,t}, \text{defend}_{x,t} \in \{0,1\} & 1 \leq t \leq T \end{cases}$$

Figure 2: The same 0-1 integer program, with burn in place of b and defend in place of d.

1.1.6 How do we solve ILPs

In many cases, similar ways to how we solve CPs – branch and bound is popular.

But there's an additional bit that we can take advantage of: linear relaxations. A linear relaxation of an integer program is a version of the program where we do not require the variables to be integers (they could instead be real numbers).

Why would we do this?

- Solving ILPs is NP-hard.
- Solving this kind of LPs is polynomial-time.

Why does this not solve our problem?

- Rounding the relaxation doesn't necessarily give us a solution (the difference is something called the **integrality gap**).

So why is it useful at all?

- The solution to a relaxation can tell us things about possible solutions to the original ILP.

In particular, the solution to the relaxation is *as good or better* than the best solution to the ILP: if it's a maximisation it's the same or bigger, if it's a minimisation it's the same or smaller.

Let's look at a picture from Wikipedia:

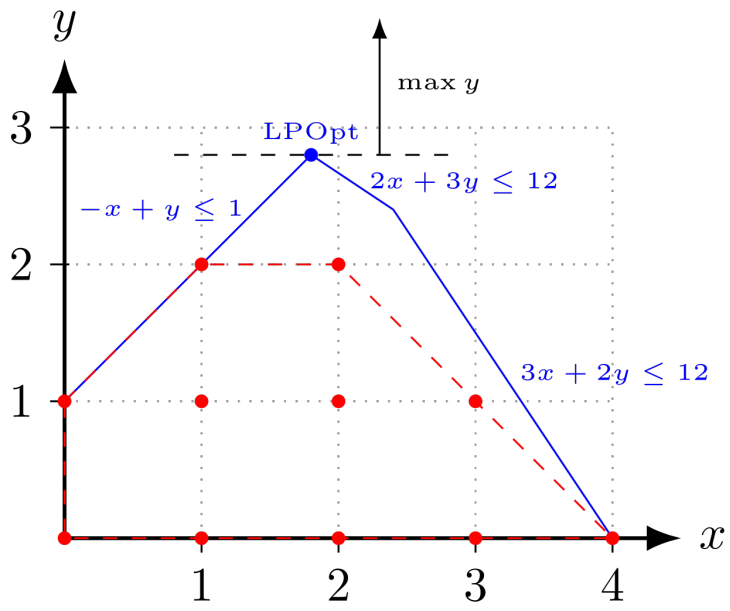


Figure 3: Polytopes of all feasible integer points and of the LP relaxation to the integer linear program $\max\{\, y \mid -x + y \leq 1;\ 3x + 2y \leq 12;\ 2x + 3y \leq 12;\ x, y \in Z_+ \,\}$. *(CC-BY; Fanosta – derivative work based on "IP polytope with LP relaxation.png" by Sdo.)*

(Extra thing to note: how visible the linear nature of the constraints is here.)

This bound helps us in pruning during search.

1.2 SAT solvers

Satisfiability solvers deal specifically in solving propositional-logic expressions of the sort we've seen in our examples of, e.g., 3-CNF-SAT.

Note that one of the fastest CP solvers around currently is based on a SAT solver: Google OR-Tools CP-SAT. It does pretty well in the MiniZinc Challenge.

Various solver approaches exist, and often combination ones work out the best. We'll outline a few.

1.2.1 Davis–Putnam–Logemann–Loveland (DPLL) algorithm

```
Algorithm DPLL
    Input: A set of clauses Φ.
    Output: A truth value indicating whether Φ is satisfiable.

function DPLL(Φ)
    while there is a unit clause {l} in Φ do
        Φ ← unit-propagate(l, Φ);
    while there is a literal l that occurs pure in Φ do
        Φ ← pure-literal-assign(l, Φ);
    if Φ is empty then
        return true;
    if Φ contains an empty clause then
        return false;
    l ← choose-literal(Φ);
    return DPLL(Φ ∧ {l}) or DPLL(Φ ∧ {not(l)});
```

Figure 4: Pseudocode for the DPLL algorithm (from Wikipedia).

- Unit propagation is something we can do when a clause has only one literal – it is a *unit clause*. Then we know that literal must be satisfied, and so we can record that, remove any clause with that literal in it, and remove the negation of the literal from any clause.
- Pure literal assignment is something we do when a variable only appears as a positive or only as a negative literal. Then we can assign the value that satisfies that literal and remove any clause with it in.

1.2.2 Randomized algorithms

Randomized or probabilistic algorithms (e.g. Paturi–Pudlak–Saks–Zane) are some of the fastest on many types of SAT instance family. We'll not talk much about these, but roughly speaking they set variables in a random order and resolve from there. The maths showing that these will work is dense, but if you're very keen – this is an option for an 'explainer' video, e.g. `https://dl.acm.org/doi/10.1145/1066100.1066101`.

A few resources:

- (just for interest, not examinable in this course): `https://www.youtube.com/watch?v=lY-aJWVTN6s`
- (just for interest, not examinable in this course): description of the OR-Tools SAT-based solver(s), `https://www.youtube.com/watch?v=lmy1ddn4cyw`

1.3 How do we compare methods?

This is really a general question about how to do science around experiments for algorithms. There is only guidance here.

First, it's a good idea to use an existing benchmark set, or think carefully about how you're generating your instances.

- Almost all instances of hard problems are easy.
- Most meaningful comparisons of approaches show strengths and weaknesses of each approach, and this often depends on the instances.

Next (if we assume they're getting the same solutions, so we're just comparing time):

- we need to do timings on the same machine;
- there are lots of weird complications about timings, mostly beyond scope here;
- we need to account for timeouts somehow.

Here's an example of a timing figure that I think is a good template:

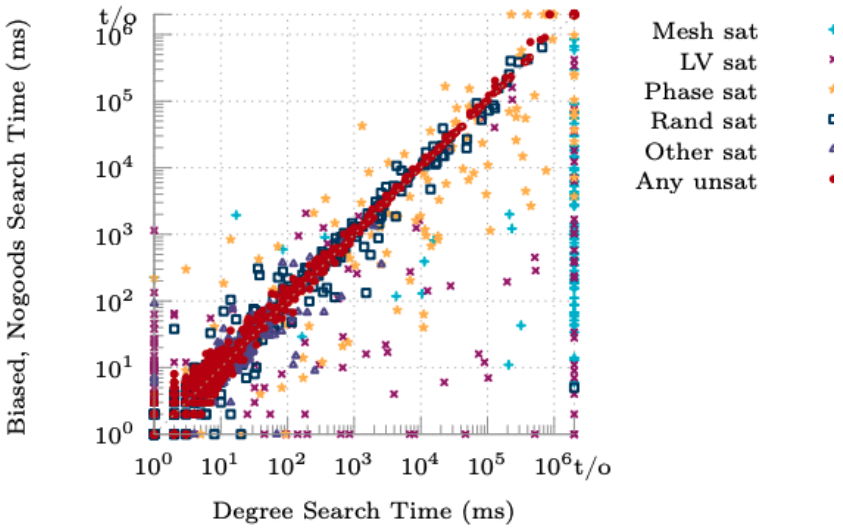


Figure 5: Degree search time versus biased, nogoods search time, across several instance families (log-log axes; `t/o` marks the timeout boundary).